

WEEK – 3 ASSIGNMENT

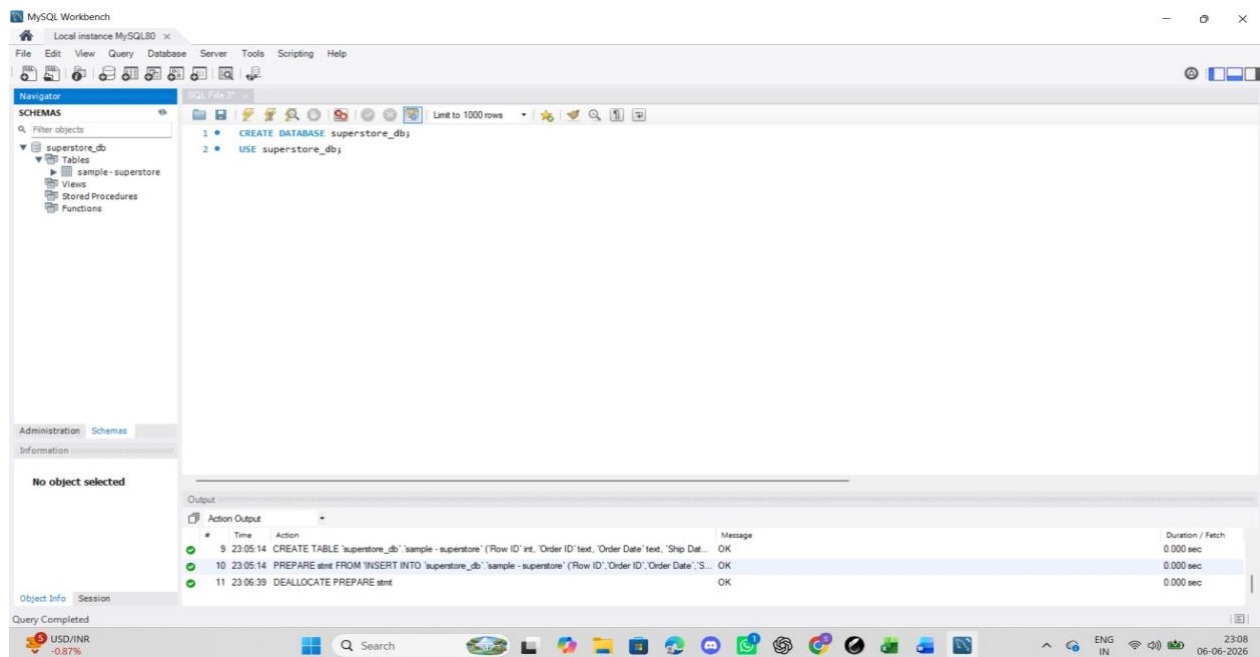
MADE BY: SHAURYANSH CHAUHAN (MMDU)

Objective:

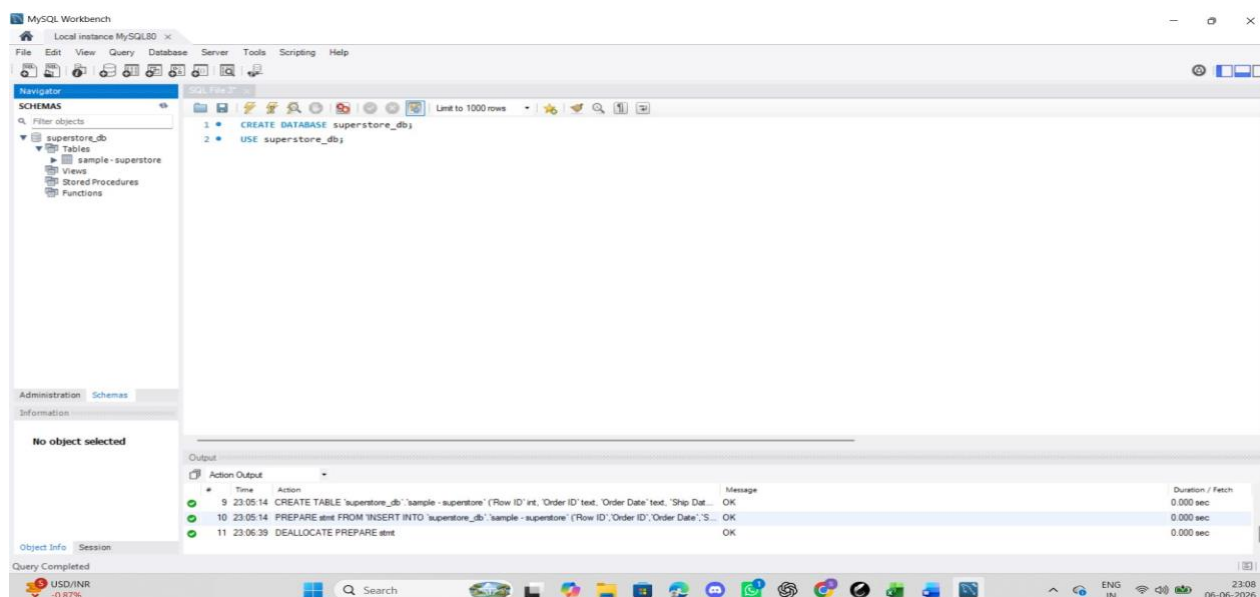
Use Subqueries, CTEs, and Window Functions to analyze sales data from the Superstore dataset.

Tasks

1 : creating database



Step 2 : Import the CSV file ile from the pc



Step 3 : Check the Imported Data

The screenshot shows the MySQL Workbench interface with the 'sample - superstore' table selected in the Navigator. The SQL editor contains the following queries:

```
1 CREATE DATABASE superstore_db;
2 USE superstore_db;
3
4 SHOW TABLES;
5
6 SELECT *
7 FROM 'sample - superstore'
8 LIMIT 10;
```

The Result Grid displays the first 10 rows of the 'sample - superstore' table. The columns are: Row ID, Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment, Country, City, State, Postal Code, Region, Product ID, Category, and Sub-Category. The data shows various orders from different customers across different regions.

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID	Category	Sub-Category
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gule	Consumer	United States	Henderson	Kentucky	42420	South	FUR-BO-10001798	Furniture	Bookcase
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gule	Consumer	United States	Henderson	Kentucky	42420	South	FUR-CH-10000454	Furniture	Chairs
3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036	West	OFF-LA-10000240	Office Supplies	Labels
4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-10000577	Furniture	Tables
5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	OFF-ST-10000760	Office Supplies	Storage
6	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California	90032	West	FUR-FU-10001487	Furniture	Furniture
7	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California	90032	West	OFF-AR-10002833	Office Supplies	Art
8	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California	90032	West	TPC-PA-10002775	Technology	Phones

The Action Output shows the execution of the queries:

- 23 23:13:35 USE superstore_db: 0 row(s) affected
- 23 23:13:52 SHOW TABLES: 1 row(s) returned
- 24 23:14:32 SELECT * FROM 'sample - superstore' LIMIT 10: 10 row(s) returned

Step 3 : Create customers table

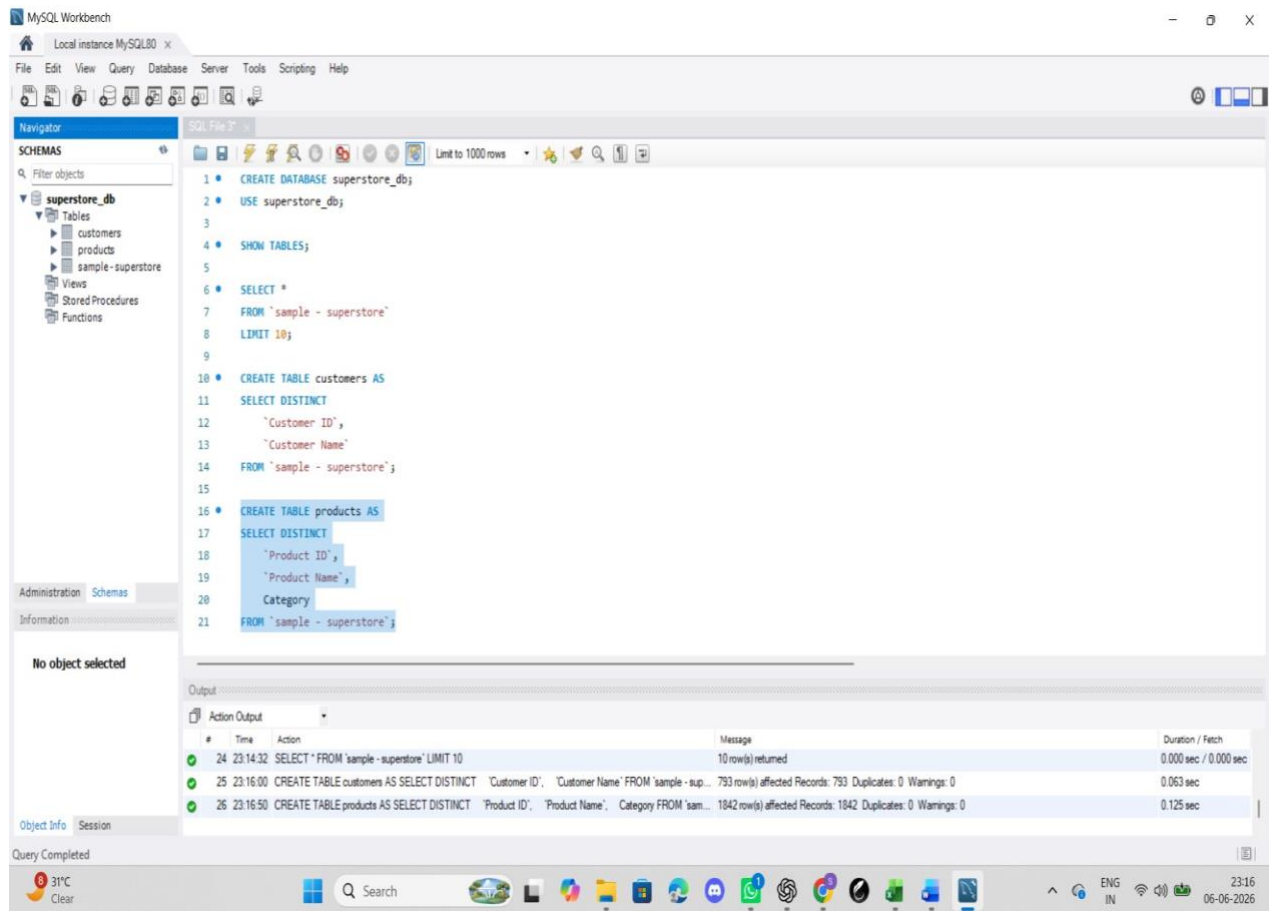
The screenshot shows the MySQL Workbench interface with the 'sample - superstore' table selected in the Navigator. The SQL editor contains the following queries:

```
1 CREATE DATABASE superstore_db;
2 USE superstore_db;
3
4 SHOW TABLES;
5
6 SELECT *
7 FROM 'sample - superstore'
8 LIMIT 10;
9
10 CREATE TABLE customers AS
11 SELECT DISTINCT
12 'Customer ID',
13 'Customer Name'
14 FROM 'sample - superstore';
```

The Action Output shows the execution of the queries:

- 23 23:13:52 SHOW TABLES: 1 row(s) returned
- 24 23:14:32 SELECT * FROM 'sample - superstore' LIMIT 10: 10 row(s) returned
- 25 23:16:00 CREATE TABLE customers AS SELECT DISTINCT 'Customer ID', 'Customer Name' FROM 'sample - superstore': 793 row(s) affected, 793 records, 0 duplicates, 0 warnings

Step 4: Create products table



MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

superstore_db

Tables

customers

products

sample-superstore

Views

Stored Procedures

Functions

Administration Schemas

Information

No object selected

Object Info Session

Query Completed

31°C Clear

SQL File 3'

Limit to 1000 rows

```
1 CREATE DATABASE superstore_db;
2 USE superstore_db;
3
4 SHOW TABLES;
5
6 SELECT *
7 FROM 'sample - superstore'
8 LIMIT 10;
9
10 CREATE TABLE customers AS
11 SELECT DISTINCT
12 'Customer ID',
13 'Customer Name'
14 FROM 'sample - superstore';
15
16 CREATE TABLE products AS
17 SELECT DISTINCT
18 'Product ID',
19 'Product Name',
20 Category
21 FROM 'sample - superstore';
```

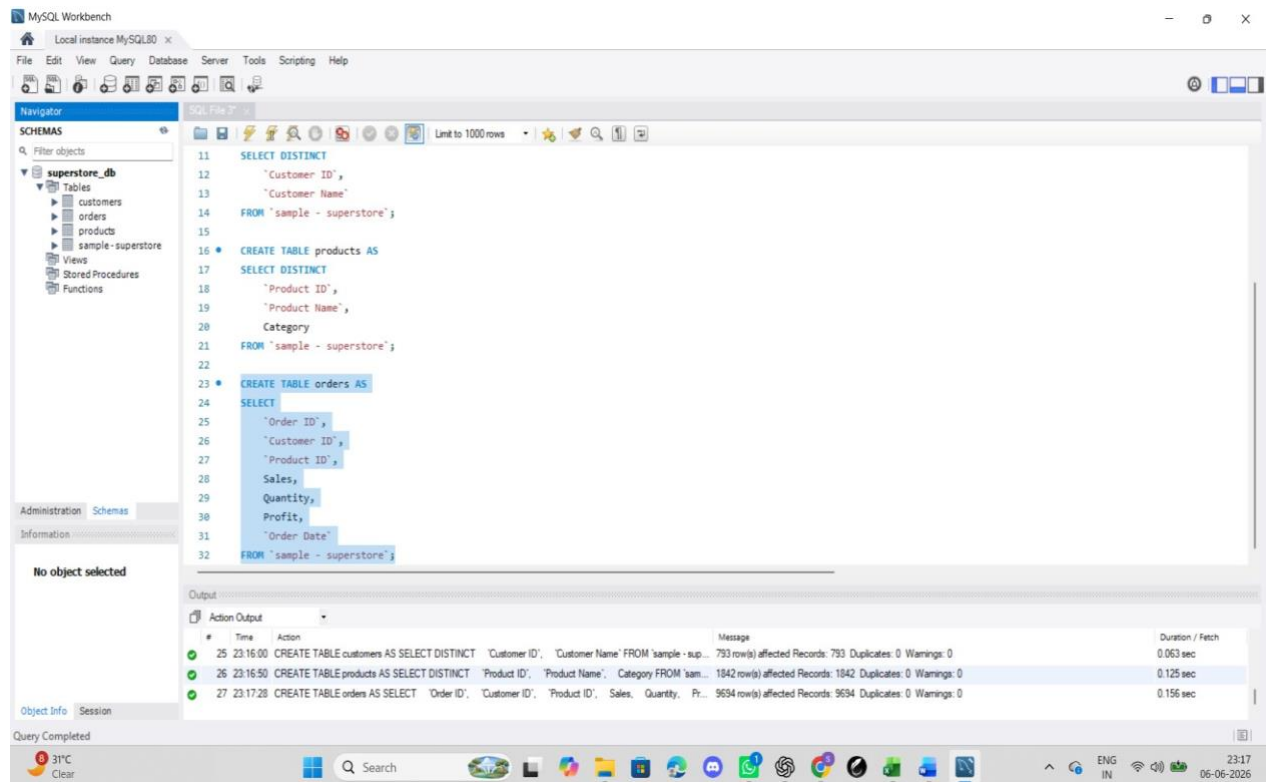
Output

Action Output

#	Time	Action	Message	Duration / Fetch
24	23:14:32	SELECT * FROM 'sample - superstore' LIMIT 10	10 row(s) returned	0.000 sec / 0.000 sec
25	23:16:00	CREATE TABLE customers AS SELECT DISTINCT 'Customer ID', 'Customer Name' FROM 'sample - superstore'	793 row(s) affected Records: 793 Duplicates: 0 Warnings: 0	0.063 sec
26	23:16:50	CREATE TABLE products AS SELECT DISTINCT 'Product ID', 'Product Name', Category FROM 'sample - superstore'	1842 row(s) affected Records: 1842 Duplicates: 0 Warnings: 0	0.125 sec

ENG IN 23:16 06-06-2026

Step 5 : Create orders table



MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

superstore_db

Tables

customers

orders

products

sample-superstore

Views

Stored Procedures

Functions

Administration Schemas

Information

No object selected

Object Info Session

Query Completed

31°C Clear

SQL File 3'

Limit to 1000 rows

```
11 SELECT DISTINCT
12 'Customer ID',
13 'Customer Name'
14 FROM 'sample - superstore';
15
16 CREATE TABLE products AS
17 SELECT DISTINCT
18 'Product ID',
19 'Product Name',
20 Category
21 FROM 'sample - superstore';
22
23 CREATE TABLE orders AS
24 SELECT
25 'Order ID',
26 'Customer ID',
27 'Product ID',
28 Sales,
29 Quantity,
30 Profit,
31 'Order Date'
32 FROM 'sample - superstore';
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
25	23:16:00	CREATE TABLE customers AS SELECT DISTINCT 'Customer ID', 'Customer Name' FROM 'sample - superstore'	793 row(s) affected Records: 793 Duplicates: 0 Warnings: 0	0.063 sec
26	23:16:50	CREATE TABLE products AS SELECT DISTINCT 'Product ID', 'Product Name', Category FROM 'sample - superstore'	1842 row(s) affected Records: 1842 Duplicates: 0 Warnings: 0	0.125 sec
27	23:17:28	CREATE TABLE orders AS SELECT 'Order ID', 'Customer ID', 'Product ID', Sales, Quantity, Profit, 'Order Date' FROM 'sample - superstore'	9694 row(s) affected Records: 9694 Duplicates: 0 Warnings: 0	0.156 sec

ENG IN 23:17 06-06-2026

Subquery – 1.Orders above average sales

The screenshot shows the MySQL Workbench interface with a query editor and a result grid. The query is as follows:

```
SELECT
  'Order ID',
  'Customer ID',
  'Product ID',
  Sales,
  Quantity,
  Profit,
  'Order Date'
FROM 'sample - superstore';
```

The result grid displays the following data:

Order ID	Customer ID	Product ID	Sales	Quantity	Profit	Order Date
CA-2016-152156	CG-12520	FUR-BO-10001798	261.96	2	41.9136	11/8/2016
CA-2016-152156	CG-12520	FUR-CH-10000454	731.94	3	219.582	11/8/2016
US-2015-108966	SO-20335	FUR-TA-10000577	957.5775	5	-383.031	10/11/2015
CA-2014-115812	BH-11710	TEC-PH-1000	90.7152	6	6/9/2014	6/9/2014
CA-2014-115812	BH-11710	FUR-TA-10001539	1706.184	9	85.3092	6/9/2014
CA-2014-115812	BH-11710	TEC-PH-10002033	911.424	4	68.3568	6/9/2014
CA-2016-161389	IM-15070	OFF-ET-10003656	407.976	3	132.5922	12/5/2016
CA-2014-105893	PK-19075	OFF-ST-10004186	665.88	6	13.3176	11/11/2014
CA-2016-106120	FR-11870	FUR-TA-10000177	1044.61	1	740.2649	9/25/2015

The output pane shows the following actions:

- 26 23:16:50 CREATE TABLE products AS SELECT DISTINCT 'Product ID', 'Product Name', 'Category' FROM 'sam... 1842 row(s) affected Records: 1842 Duplicates: 0 Warnings: 0 0.125 sec
- 27 23:17:28 CREATE TABLE orders AS SELECT 'Order ID', 'Customer ID', 'Product ID', Sales, Quantity, Pr... 9634 row(s) affected Records: 9634 Duplicates: 0 Warnings: 0 0.156 sec
- 28 23:18:25 SELECT * FROM orders WHERE Sales > (SELECT AVG(Sales) FROM orders) LIMIT 0, 1000 1000 row(s) returned 0.000 sec / 0.000 sec

Subquery – 2.Highest order for each customer

The screenshot shows the MySQL Workbench interface with a query editor and a result grid. The query is as follows:

```
FROM orders
))
SELECT *
FROM orders o
WHERE Sales =
(
  SELECT MAX(Sales)
  FROM orders
  WHERE 'Customer ID' = o.'Customer ID'
);
```

The result grid displays the following data:

Order ID	Customer ID	Product ID	Sales	Quantity	Profit	Order Date
CA-2016-152156	CG-12520	FUR-CH-10000454	731.94	3	219.582	11/8/2016
US-2015-108966	SO-20335	FUR-TA-10000577	957.5775	5	-383.031	10/11/2015
CA-2014-115812	BH-11710	FUR-TA-10001539	1706.184	9	85.3092	6/9/2014
CA-2015-106320	EB-13870	FUR-TA-10000577	1044.63	3	240.2649	9/25/2015
US-2015-150630	TB-21520	FUR-BO-10004834	3083.43	7	-1665.0522	9/17/2015
CA-2016-117590	GH-14485	TEC-PH-10004977	1097.544	7	123.4737	12/8/2016
CA-2015-117415	SN-20710	FUR-BO-10002545	532.3992	3	-46.9764	12/27/2015
CA-2016-105816	JM-15265	TEC-PH-10002447	1029.95	5	298.6855	12/11/2016
CA-2016-111687	TR-71055	FUR-CH-10001968	119.41	5	7.098	6/17/2016

The output pane shows the following actions:

- 27 23:17:28 CREATE TABLE orders AS SELECT 'Order ID', 'Customer ID', 'Product ID', Sales, Quantity, ... 9634 row(s) affected Records: 9634 Duplicates: 0 Warnings: 0 0.156 sec
- 28 23:18:25 SELECT * FROM orders WHERE Sales > (SELECT AVG(Sales) FROM orders) LIMIT 0, 1000 1000 row(s) returned 0.000 sec / 0.000 sec
- 29 23:19:02 SELECT * FROM orders o WHERE Sales = (SELECT MAX(Sales) FROM orders WHERE 'Customer ID' = o.'Customer ID' 755 row(s) returned 21.281 sec / 63.187 sec

Step CTE – Total sales per customer

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
WITH customer_sales AS
(
    SELECT
        "Customer ID",
        SUM(Sales) AS Total_Sales
    FROM orders
    GROUP BY "Customer ID"
)
SELECT *
FROM customer_sales;
```

The Result Grid displays the following data:

Customer ID	Total_Sales
CG-12520	1148.7800000000002
DV-13045	1119.4630000000002
SO-20335	2602.5750000000001
BH-11710	6255.3510000000001
AA-10480	1782.872
IM-15070	4930.4739999999999
HP-14815	1990.314
PK-19075	8158.654
AG-10730	7561.7579999999994

The Action Output pane shows the execution progress of the query.

Step window function – ROW_NUMBER()

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
FROM customer_sales;
SELECT
    "Customer ID",
    Sales,
    ROW_NUMBER() OVER
    (
        PARTITION BY "Customer ID"
        ORDER BY Sales DESC
    ) AS Row_Num
FROM orders;
```

The Result Grid displays the following data:

Customer ID	Sales	Row_Num
AA-10315	3930.072	1
AA-10315	673.568	2
AA-10315	431.976	3
AA-10315	362.94	4
AA-10315	52.98	5
AA-10315	41.72	6
AA-10315	26.96	7
AA-10315	14.94	8
AA-10315	14.56	9

The Action Output pane shows the execution progress of the query.

Step 2 window function – RANK()

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
SELECT
  'Customer ID',
  SUM(Sales) AS Total_Sales,
  RANK() OVER
  (
    ORDER BY SUM(Sales) DESC
  ) AS Customer_Rank
FROM orders
GROUP BY 'Customer ID';
```

The Result Grid displays the following data:

Customer ID	Total_Sales	Customer_Rank
SM-20330	25043.05	1
TC-20980	19017.847999999998	2
RB-19360	15117.339	3
TA-21385	14595.62	4
AB-10105	14355.610999999997	5
SC-20095	14142.333999999999	6
KL-16645	14071.917	7
HL-15040	12873.297999999999	8
GF-20110	12209.438000000002	9

The Output pane shows the execution log with the following messages:

- 30 23:21:23 WITH customer_sales AS (SELECT 'Customer ID', SUM(Sales) AS Total_Sales FROM orders... 793 row(s) returned 0.015 sec / 0.000 sec
- 31 23:22:08 SELECT 'Customer ID', Sales, ROW_NUMBER() OVER (PARTITION BY 'Customer ID' ... 9694 row(s) returned 0.015 sec / 0.000 sec
- 32 23:22:50 SELECT 'Customer ID', SUM(Sales) AS Total_Sales, RANK() OVER (ORDER BY SUM(Sale... 793 row(s) returned 0.063 sec / 0.000 sec

Step Final query (JOIN + CTE + Window Function)

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
WITH customer_sales AS
(
  SELECT
    'Customer ID',
    SUM(Sales) AS Total_Sales
  FROM orders
  GROUP BY 'Customer ID'
)
SELECT
  Customer ID,
  Total_Sales,
  RANK() OVER
  (
    ORDER BY SUM(Sale...
```

The Result Grid displays the following data:

Customer ID	Total_Sales	Sales_Rank
Sean Miller	25043.05	1
Tamara Chand	19017.847999999998	2
Raymond Buch	15117.339	3
Tom Ashbrook	14595.62	4
Adrian Barton	14355.610999999997	5
Sargit Chand	14142.333999999999	6
Ken Lonsdale	14071.917	7
Hunter Lopez	12873.297999999999	8
Sargit Engle	12209.438000000002	9

The Output pane shows the execution log with the following messages:

- 31 23:22:08 SELECT 'Customer ID', Sales, ROW_NUMBER() OVER (PARTITION BY 'Customer ID' ... 9694 row(s) returned 0.015 sec / 0.000 sec
- 32 23:22:50 SELECT 'Customer ID', SUM(Sales) AS Total_Sales, RANK() OVER (ORDER BY SUM(Sale... 793 row(s) returned 0.063 sec / 0.000 sec
- 33 23:23:43 WITH customer_sales AS (SELECT 'Customer ID', SUM(Sales) AS Total_Sales FROM orders... 793 row(s) returned 0.031 sec / 0.000 sec

Mini Project

Q1.The top 5 Customers

The screenshot shows MySQL Workbench with a query window open. The query is a window function that ranks customers by total sales. The results grid shows the top 5 customers.

```
WITH customer_sales AS
(
  SELECT
    "Customer ID",
    SUM(Sales) AS Total_Sales
  FROM orders
  GROUP BY "Customer ID"
)
SELECT *
FROM customer_sales
ORDER BY Total_Sales DESC
LIMIT 5;
```

Customer ID	Total_Sales
SM-20320	25043.05
TC-20980	19017.847999999998
RB-19360	15117.339
TA-21385	14595.62
AB-10105	14355.610999999997

Output:

#	Time	Action	Message	Duration / Fetch
32	23:22:50	SELECT "Customer ID", SUM(Sales) AS Total_Sales, RANK() OVER (ORDER BY SUM(Sale...	793 row(s) returned	0.063 sec / 0.000 sec
33	23:23:43	WITH customer_sales AS (SELECT "Customer ID", SUM(Sales) AS Total_Sales FROM orders...	793 row(s) returned	0.031 sec / 0.000 sec
34	23:24:31	WITH customer_sales AS (SELECT "Customer ID", SUM(Sales) AS Total_Sales FROM orders...	10 row(s) returned	0.016 sec / 0.000 sec

Q2.Bottom 5 Customers

The screenshot shows MySQL Workbench with a query window open. The query is a window function that ranks customers by total sales. The results grid shows the bottom 5 customers.

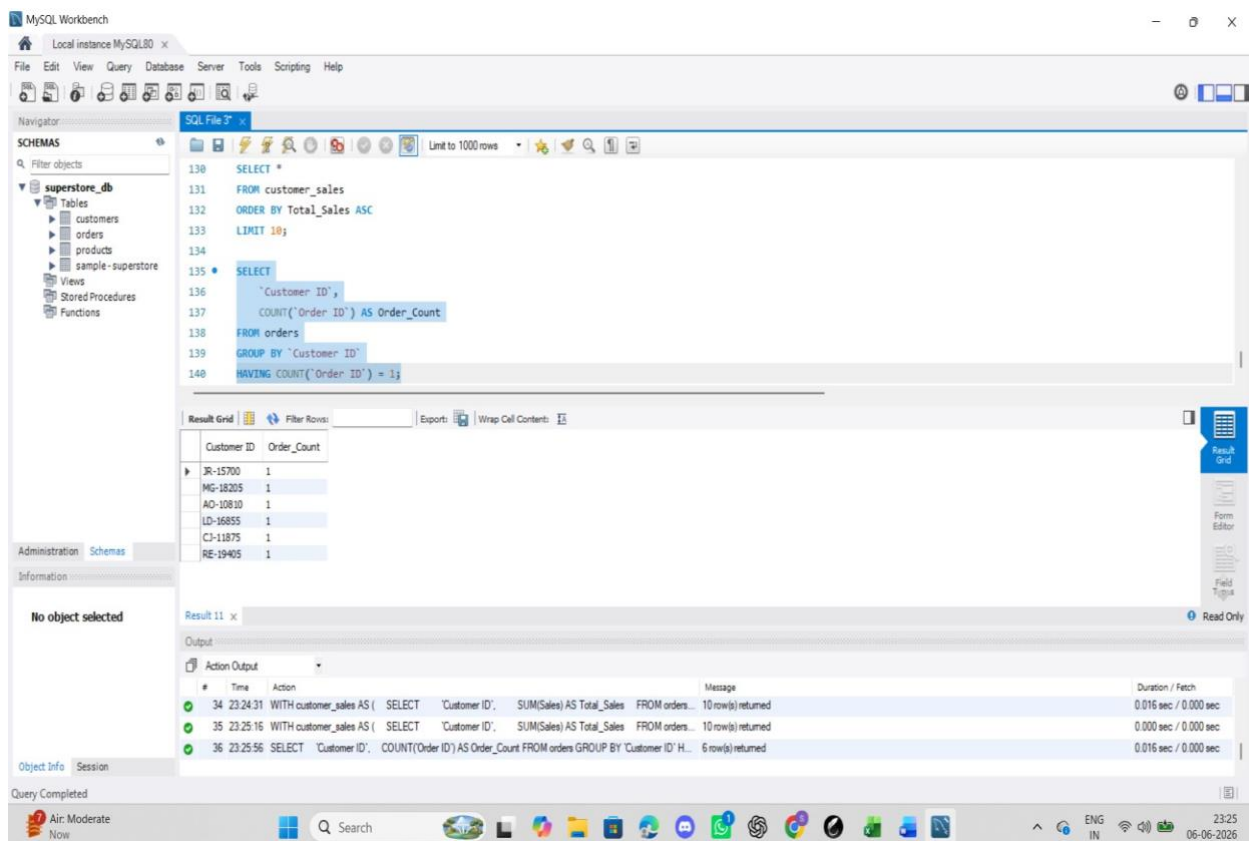
```
WITH customer_sales AS
(
  SELECT
    "Customer ID",
    SUM(Sales) AS Total_Sales
  FROM orders
  GROUP BY "Customer ID"
)
SELECT *
FROM customer_sales
ORDER BY Total_Sales ASC
LIMIT 5;
```

Customer ID	Total_Sales
TS-21085	4.833
LD-16855	5.304
MG-18205	12.32
CJ-11875	16.52
RS-19870	22.328

Output:

#	Time	Action	Message	Duration / Fetch
33	23:23:43	WITH customer_sales AS (SELECT "Customer ID", SUM(Sales) AS Total_Sales FROM orders...	793 row(s) returned	0.031 sec / 0.000 sec
34	23:24:31	WITH customer_sales AS (SELECT "Customer ID", SUM(Sales) AS Total_Sales FROM orders...	10 row(s) returned	0.016 sec / 0.000 sec
35	23:25:16	WITH customer_sales AS (SELECT "Customer ID", SUM(Sales) AS Total_Sales FROM orders...	10 row(s) returned	0.000 sec / 0.000 sec

Q3. Customers with only one order



MySQL Workbench interface showing a query for customers with only one order. The query is:

```
SELECT *
FROM customer_sales
ORDER BY Total_Sales ASC
LIMIT 10;

SELECT
  'Customer ID',
  COUNT('Order ID') AS Order_Count
FROM orders
GROUP BY 'Customer ID'
HAVING COUNT('Order ID') = 1;
```

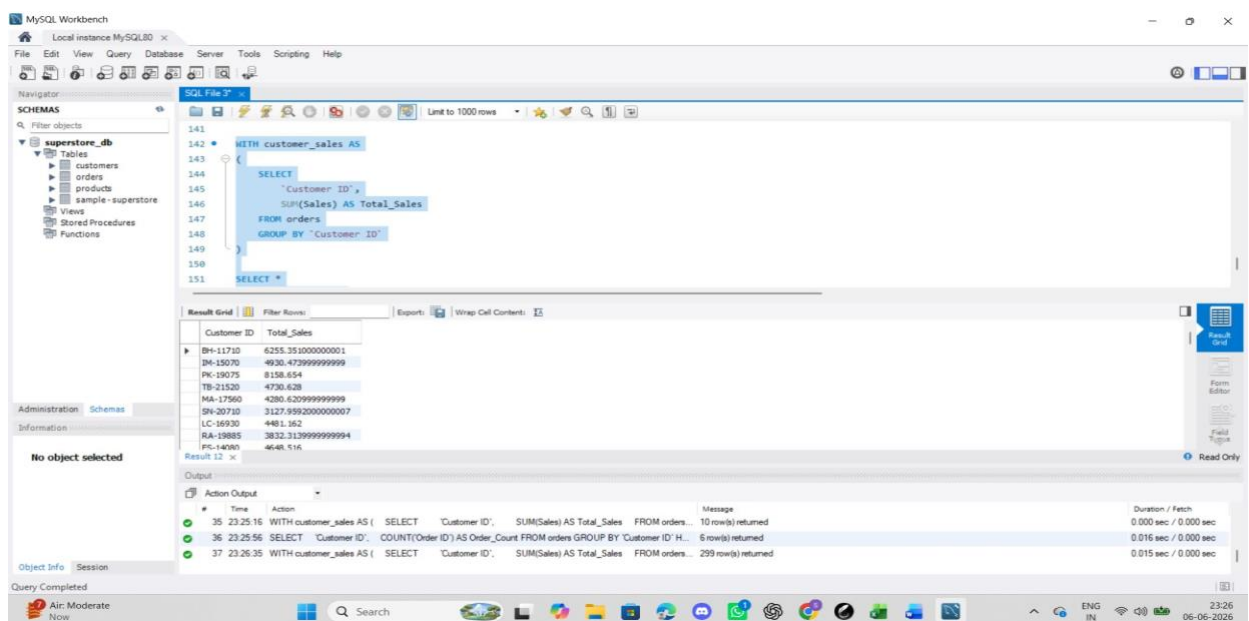
The result grid displays the following data:

Customer ID	Order_Count
JR-15700	1
MG-18205	1
AD-10810	1
LD-16855	1
CJ-11875	1
RE-19405	1

The bottom panel shows the execution log with the following messages:

- 34 23:24:31 WITH customer_sales AS (SELECT 'Customer ID', SUM(Sales) AS Total_Sales FROM orders... 10 row(s) returned 0.016 sec / 0.000 sec
- 35 23:25:16 WITH customer_sales AS (SELECT 'Customer ID', SUM(Sales) AS Total_Sales FROM orders... 10 row(s) returned 0.000 sec / 0.000 sec
- 36 23:25:56 SELECT 'Customer ID', COUNT(Order ID) AS Order_Count FROM orders GROUP BY 'Customer ID' H... 6 row(s) returned 0.016 sec / 0.000 sec

Q4. Customers above average total sales



MySQL Workbench interface showing a query for customers above average total sales. The query is:

```
WITH customer_sales AS
(
  SELECT
    'Customer ID',
    SUM(Sales) AS Total_Sales
  FROM orders
  GROUP BY 'Customer ID'
)
SELECT *
```

The result grid displays the following data:

Customer ID	Total_Sales
BH-11710	6255.351000000001
JH-15070	4930.4799999999999
PK-19075	8158.654
TB-21520	4750.628
MA-17560	4280.6299999999999
SN-20710	3127.9592000000007
LC-16930	4481.162
RA-19805	3832.3129999999994
PG-14090	4648.516

The bottom panel shows the execution log with the following messages:

- 35 23:25:16 WITH customer_sales AS (SELECT 'Customer ID', SUM(Sales) AS Total_Sales FROM orders... 10 row(s) returned 0.000 sec / 0.000 sec
- 36 23:25:56 SELECT 'Customer ID', COUNT(Order ID) AS Order_Count FROM orders GROUP BY 'Customer ID' H... 6 row(s) returned 0.016 sec / 0.000 sec
- 37 23:26:35 WITH customer_sales AS (SELECT 'Customer ID', SUM(Sales) AS Total_Sales FROM orders... 299 row(s) returned 0.015 sec / 0.000 sec

Q 5: WHAT IS THE HIGHEST ORDER VALUE PER CUSTOMER?

The screenshot shows a database management interface. On the left, a tree view displays the 'superstore_db' schema with folders for Tables, Views, Stored Procedures, Functions, and a 'sys' database. Below this, tabs for 'Administration', 'Schemas', and 'Information' are visible, with 'Schemas' selected. The main area shows a SQL query editor with the following text:

```
-- Q 5: WHAT IS THE HIGHEST ORDER VALUE PER CUSTOMER?  
  
SELECT  
    c.`Customer Name`,  
    MAX(o.Sales) AS Highest_Order_Value  
FROM customers c  
JOIN orders o  
ON c.`Customer ID` = o.`Customer ID`  
GROUP BY c.`Customer ID`, c.`Customer Name`  
ORDER BY Highest_Order_Value DESC;
```

Below the query editor, a 'Result Grid' is shown with two columns: 'Customer Name' and 'Highest_Order_Value'. The grid is currently empty. To the right of the grid, there are controls for 'Filter Rows:', 'Export:', and 'Wrap Cell Conte'.